LOGO ENSTA
(Marcador de imagen)

RO03 – Juegos, grafos e investigación operativa

Reporte explicativo del proyecto

Tracks: modelado paso a paso como problema de programación lineal entera

Autores

CHACON Jose

FLORIDO Santiago

Tema: Modelado matemático del puzzle Tracks

École Nationale des techniques avancées
Abril de 2026

Índice

1. Introducción **1**

2. Modelo Matemático Paso a Paso **1**

2.1. Enunciado del problema 2

2.2. De la grilla al grafo 2

2.3. Conjuntos, índices y parámetros 3

2.4. Variables de decisión 4

2.5. Restricciones del modelo 5

2.5.1. Restricciones de dominio 5

2.5.2. Restricciones de pistas de filas y columnas 5

2.5.3. Restricciones de coherencia entre casillas y conexiones 5

2.5.4. Restricciones de terminales 6

2.5.5. Restricciones de grado para las demás casillas 6

2.5.6. Restricciones para información fija 6

2.5.7. Restricciones de conectividad mediante flujo 7

2.6. Modelo completo 7

3. Interpretación del Modelo **8**

3.1. Por qué el modelo produce un camino 8

3.2. Qué configuraciones inválidas quedan eliminadas 8

3.2.1. Ramas 8

3.2.2. Casillas aisladas 8

3.2.3. Ciclos aislados 9

3.2.4. Componentes desconectadas 9

3.3. Relación entre las pistas y la geometría 9

3.4. Por qué no se necesitan variables de tipo de pieza 9

3.5. Comentarios sobre otras formulaciones posibles 9

3.6. Resumen final del modelado 10

1 Introducción

El objetivo de este reporte es explicar el puzzle *Tracks* desde el punto de vista de la investigación operativa. Antes de hablar de programación, de Python o de visualización con Pygame, se necesita una descripción matemática clara del problema. Esta descripción es importante porque el solver que se implementará después debe resolver exactamente el problema definido por el modelo, y no una aproximación informal de las reglas.

En Tracks se tiene una grilla rectangular. En la grilla aparecen dos terminales: un punto de salida y un punto de llegada. La solución consiste en construir una vía de tren continua que conecte esos dos terminales. Además, en el borde de la grilla aparecen números. Cada número de fila indica cuántas casillas de esa fila deben contener vía. Cada número de columna indica cuántas casillas de esa columna deben contener vía.

La dificultad del puzzle no está solamente en contar casillas. También hay que garantizar que las piezas de vía formen un único camino correcto. Por ejemplo, una solución no puede tener una rama, porque una vía de Tracks no se divide en dos. Tampoco puede tener un ciclo separado, aunque ese ciclo respete los números de filas y columnas. La solución debe ser una sola ruta que empieza en el terminal inicial y termina en el terminal final.

Esta situación se modela de forma natural con grafos. En el vocabulario del curso, un grafo se escribe normalmente como

$$G = (V, E),$$

donde V es el conjunto de vértices y E es el conjunto de aristas. Para Tracks, los vértices serán las casillas de la grilla, y las aristas serán las posibles conexiones entre casillas vecinas. Así, una vía de tren se convierte en un subconjunto de vértices y aristas del grafo.

El modelo que se presenta en este reporte es un modelo de factibilidad. Esto significa que no buscamos minimizar un coste real ni maximizar una ganancia. Buscamos responder a la pregunta:

¿existe una solución que respete todas las reglas del puzzle?

Para escribirlo como un modelo lineal entero mixto, se puede usar una función objetivo constante, por ejemplo $\min 0$. En ese caso, el solver no intenta mejorar un valor numérico; simplemente intenta encontrar una solución que satisfaga todas las restricciones.

El reporte está escrito de manera progresiva. Primero se explica cómo pasar de la grilla a un grafo. Después se definen los conjuntos, los parámetros y las variables de decisión. Luego se escriben las restricciones una por una, explicando qué regla del puzzle representa cada bloque. Finalmente, se interpreta el modelo completo y se explica por qué elimina configuraciones inválidas como ramas, ciclos aislados o componentes desconectadas.

> **Idea principal.** Tracks se puede ver como un problema de selección de subgrafo. Elegimos qué casillas se usan y qué conexiones entre casillas se activan. Las restricciones obligan a que esa selección forme un único camino desde el terminal inicial hasta el terminal final.

2 Modelo Matemático Paso a Paso

2.1 Enunciado del problema

Consideramos una instancia de *Tracks* definida sobre una grilla rectangular de m filas y n columnas. Hay dos casillas especiales:

$$s \quad \text{y} \quad t.$$

La casilla s representa el terminal de inicio, y la casilla t representa el terminal de llegada. La solución debe construir una vía de tren que conecte s con t.

Para que una configuración sea válida, debe cumplir todas las reglas siguientes:

- la vía debe empezar en s y terminar en t;
- cada casilla usada debe formar parte de la vía;
- las conexiones entre casillas deben ser coherentes con una vía de tren;
- la vía no puede ramificarse;
- no puede haber piezas sueltas ni componentes desconectadas;
- no puede haber ciclos aislados que no estén conectados con los terminales;
- cada fila debe usar exactamente el número de casillas indicado por su pista;
- cada columna debe usar exactamente el número de casillas indicado por su pista;
- las casillas pre-rellenadas o fijas deben respetarse.

Desde el punto de vista de optimización, este problema es un problema de factibilidad. La palabra "factibilidad" significa que queremos saber si existe al menos una solución que cumpla las restricciones. Por eso se puede escribir el modelo con el objetivo artificial

$$\text{mín } 0.$$

Esta expresión no tiene un significado económico. Simplemente permite poner el problema dentro de la forma estándar de un modelo de programación lineal entera mixta.

2.2 De la grilla al grafo

El primer paso consiste en transformar la grilla en un grafo. Esto permite usar conceptos vistos en teoría de grafos: vértice, arista, grado, camino, ciclo y componente conexa.

Cada casilla de la grilla se convierte en un vértice. Si dos casillas son vecinas horizontal o verticalmente, entonces se crea una arista entre los dos vértices correspondientes. Las casillas diagonales no son vecinas, porque una vía de Tracks solo puede pasar de una casilla a otra por los lados.

Por ejemplo, si una casilla está en la posición (i, j), sus posibles vecinos son:

$$(i-1, j), \quad (i+1, j), \quad (i, j-1), \quad (i, j+1),$$

siempre que esas posiciones existan dentro de la grilla. Una casilla de esquina tiene menos vecinos que una casilla interior, porque está en el borde.

El grafo de la grilla se escribe

$$G = (V, E),$$

donde V es el conjunto de casillas y E es el conjunto de conexiones posibles entre casillas vecinas.

Interpretación. Resolver Tracks equivale a elegir algunos vértices y algunas aristas de G. Los vértices elegidos son las casillas con vía. Las aristas elegidas son las conexiones reales entre esas casillas.

2.3 Conjuntos, índices y parámetros

Definimos el conjunto de filas como

$$R = \{1, \ldots, m\},$$

y el conjunto de columnas como

$$C = \{1, \ldots, n\}.$$

Entonces el conjunto de todas las casillas de la grilla es

$$V = R \times C.$$

Esto significa que una casilla es un par (i, j), donde i indica la fila y j indica la columna.

El conjunto de aristas se divide en aristas horizontales y verticales:

$$E = E^H \cup E^V.$$

Las aristas horizontales son

$$E^H = \{\{(i, j), (i, j + 1)\} : i \in R,\ j \in \{1, \ldots, n - 1\}\}.$$

Esta fórmula dice que hay una arista horizontal entre una casilla y la casilla que está justo a su derecha, siempre que esa casilla exista.

Las aristas verticales son

$$E^V = \{\{(i, j), (i + 1, j)\} : i \in \{1, \ldots, m - 1\},\ j \in C\}.$$

Esta fórmula dice que hay una arista vertical entre una casilla y la casilla que está justo debajo, siempre que esa casilla exista.

Para una casilla $v \in V$, denotamos por $N(v)$ el conjunto de vecinos de v. También denotamos por

$$\delta(v) = \{e \in E : v \in e\}$$

el conjunto de aristas incidentes a v. En otras palabras, $\delta(v)$ contiene todas las aristas que tocan la casilla v. El número de aristas seleccionadas dentro de $\delta(v)$ será el grado de la casilla en la solución.

También usaremos notación para subconjuntos de casillas. Si $S \subseteq V$, entonces

$$E(S) = \{\{u, v\} \in E : u \in S,\ v \in S\}$$

es el conjunto de aristas internas a S, y

$$\delta(S) = \{\{u, v\} \in E : u \in S,\ v \notin S\}$$

es el conjunto de aristas que salen de S. Esta última notación es útil para hablar de conectividad, porque una componente está separada del resto del grafo cuando no hay aristas seleccionadas que salgan de ella.

Los terminales se denotan por

$$s \in V, \qquad t \in V, \qquad s \neq t.$$

El terminal s es el inicio de la vía, y el terminal t es el final.

Las pistas de filas se denotan por

$$\rho_i \in \mathbb{Z}_+ \qquad \forall i \in R,$$

y las pistas de columnas por

$$\gamma_j \in \mathbb{Z}_+ \qquad \forall j \in C.$$

El número ρ_i indica cuántas casillas de la fila i deben contener vía. El número γ_j indica cuántas casillas de la columna j deben contener vía.

Como las pistas de filas y columnas cuentan las mismas casillas, se debe cumplir la condición necesaria

$$\sum_{i \in R} \rho_i = \sum_{j \in C} \gamma_j.$$

Si esta igualdad falla, entonces la instancia no puede tener solución, porque las filas y las columnas estarían pidiendo cantidades totales diferentes de casillas con vía.

Para representar información fija, definimos:

$$V^+ \subseteq V$$

como el conjunto de casillas obligadas a tener vía, y

$$V^- \subseteq V$$

como el conjunto de casillas obligadas a estar vacías. Además, si algunas casillas tienen una forma de vía pre-rellenada, usamos el conjunto $\mathcal{P}$. Para cada $v \in \mathcal{P}$, el conjunto

$$P_v \subseteq \delta(v)$$

indica qué aristas incidentes deben estar activadas.

2.4 Variables de decisión

La primera familia de variables indica si una casilla se usa o no. Para cada $v \in V$, definimos

$$y_v = \begin{cases} 1 & \text{si la casilla } v \text{ contiene vía,} \\ 0 & \text{si la casilla } v \text{ está vacía.} \end{cases}$$

Estas variables son útiles para contar cuántas casillas se usan en cada fila y en cada columna.

La segunda familia de variables indica si una conexión entre dos casillas vecinas se usa o no. Para cada arista $e \in E$, definimos

$$x_e = \begin{cases} 1 & \text{si la conexión } e \text{ forma parte de la vía,} \\ 0 & \text{si la conexión } e \text{ no se usa.} \end{cases}$$

Estas variables describen la geometría real de la vía. Por ejemplo, si dos casillas vecinas contienen vía pero no hay conexión entre ellas, entonces esas dos piezas no están unidas.

Finalmente, para imponer que todo esté conectado, se introducen variables auxiliares de flujo. Primero se reemplaza cada arista no dirigida por dos arcos dirigidos. El conjunto de arcos es

$$A = \{(u, v) \in V \times V : \{u, v\} \in E\}.$$

Para cada arco $(u, v) \in A$, definimos una variable continua

$$f_{uv} \geq 0.$$

Esta variable no representa un tren real. Es una herramienta matemática. La idea es enviar flujo desde el terminal inicial s hacia todas las casillas usadas. Si una casilla usada no puede recibir flujo desde s, entonces esa casilla está desconectada y la solución no es válida.

2.5 Restricciones del modelo

2.5.1 Restricciones de dominio

Las variables de casilla y de arista son binarias:

$$y_v \in \{0, 1\} \qquad \forall v \in V. \tag{1}$$

$$x_e \in \{0, 1\} \qquad \forall e \in E. \tag{2}$$

Esto significa que no existe una casilla "medio usada" ni una conexión "medio seleccionada".

Las variables de flujo son continuas y no negativas:

$$f_{uv} \geq 0 \qquad \forall (u, v) \in A. \tag{3}$$

2.5.2 Restricciones de pistas de filas y columnas

Para cada fila i, la suma de las casillas usadas debe ser igual a la pista ρ_i:

$$\sum_{j \in C} y_{(i,j)} = \rho_i \qquad \forall i \in R. \tag{4}$$

Esta ecuación dice: "en la fila i, exactamente ρ_i casillas deben contener vía".

Para cada columna j, la suma de las casillas usadas debe ser igual a la pista γ_j:

$$\sum_{i \in R} y_{(i,j)} = \gamma_j \qquad \forall j \in C. \tag{5}$$

Esta ecuación dice lo mismo, pero para columnas. Así se traducen directamente las pistas numéricas del puzzle.

2.5.3 Restricciones de coherencia entre casillas y conexiones

Una conexión solo puede existir si las dos casillas que conecta están usadas. Para cada arista $\{u, v\} \in E$, imponemos

$$x_{\{u,v\}} \leq y_u, \tag{6}$$

y también

$$x_{\{u,v\}} \leq y_v. \tag{7}$$

La primera desigualdad impide usar la conexión si u está vacío. La segunda impide usarla si v está vacío. Por tanto, una arista seleccionada siempre une dos casillas con vía.

2.5.4 Restricciones de terminales

Los terminales deben estar usados:

$$y_s = 1, \qquad y_t = 1. \tag{8}$$

Esto es obligatorio porque la vía debe empezar en s y terminar en t.

Además, cada terminal debe tener exactamente una conexión seleccionada:

$$\sum_{e \in \delta(s)} x_e = 1, \qquad \sum_{e \in \delta(t)} x_e = 1. \tag{9}$$

Esta ecuación dice que de s sale una única vía y que a t llega una única vía. Si hubiera dos conexiones en un terminal, el terminal sería una pieza intermedia, no un extremo.

2.5.5 Restricciones de grado para las demás casillas

Para toda casilla que no sea terminal, se impone

$$\sum_{e \in \delta(v)} x_e = 2y_v \qquad \forall v \in V \setminus \{s, t\}. \tag{10}$$

Esta ecuación es una de las más importantes del modelo.

Si $y_v = 0$, entonces el lado derecho vale 0, y por tanto ninguna arista incidente puede estar seleccionada. La casilla está vacía y no se conecta a nada.

Si $y_v = 1$, entonces el lado derecho vale 2, y exactamente dos aristas incidentes deben estar seleccionadas. Eso significa que una casilla usada no terminal tiene una entrada y una salida. Puede ser una pieza recta o una curva, pero no puede ser una rama.

Esta restricción elimina muchas configuraciones inválidas. Una rama necesitaría una casilla con tres conexiones. Un cruce necesitaría cuatro conexiones. Una casilla usada aislada tendría cero conexiones. Todas esas situaciones violan (10).

2.5.6 Restricciones para información fija

Si una casilla está fijada como usada, imponemos

$$y_v = 1 \qquad \forall v \in V^+. \tag{11}$$

Si una casilla está fijada como vacía, imponemos

$$y_v = 0 \qquad \forall v \in V^-. \tag{12}$$

Si una casilla tiene una forma de vía pre-rellenada, se fijan las aristas correspondientes. Para cada $v \in \mathcal{P}$, se impone

$$x_e = 1 \qquad \forall e \in P_v, \tag{13}$$

y

$$x_e = 0 \qquad \forall e \in \delta(v) \setminus P_v. \tag{14}$$

Así, el modelo no puede ignorar las piezas ya dadas por la instancia.

2.5.7 Restricciones de conectividad mediante flujo

Las restricciones de grado garantizan una buena estructura local, pero no garantizan por sí solas que toda la vía sea una sola pieza. Todavía podría aparecer un camino correcto de s a t más un ciclo separado en otra zona de la grilla. Ese ciclo podría respetar los grados y las pistas, pero no sería válido en Tracks.

Para evitarlo, usamos una formulación de flujo. El terminal inicial s envía una unidad de flujo a cada casilla usada distinta de s. El flujo solo puede pasar por aristas seleccionadas.

Tomamos

$$M = |V| - 1.$$

Este valor es suficientemente grande porque, como máximo, hay $|V| - 1$ casillas distintas de s que pueden necesitar flujo.

La capacidad del flujo se escribe así:

$$0 \leq f_{uv} \leq M x_{\{u,v\}} \qquad \forall (u,v) \in A. \tag{15}$$

Esta desigualdad dice que si la arista $\{u,v\}$ no está seleccionada, entonces $x_{\{u,v\}} = 0$, y por tanto $f_{uv} = 0$. Es decir, no puede pasar flujo por una conexión que no forma parte de la vía.

En el terminal inicial s, el flujo neto que sale debe ser igual al número de casillas usadas distintas de s:

$$\sum_{u \in N(s)} f_{su} - \sum_{u \in N(s)} f_{us} = \sum_{v \in V \setminus \{s\}} y_v. \tag{16}$$

Esta ecuación dice que s es la fuente del flujo.

Para toda casilla $v \neq s$, imponemos

$$\sum_{u \in N(v)} f_{uv} - \sum_{u \in N(v)} f_{vu} = y_v \qquad \forall v \in V \setminus \{s\}. \tag{17}$$

Si v está usada, entonces $y_v = 1$, y la casilla debe recibir una unidad neta de flujo. Si v está vacía, entonces $y_v = 0$, y no debe consumir flujo.

Estas restricciones eliminan componentes desconectadas. Si un grupo de casillas usadas estuviera separado de s, no habría ninguna arista seleccionada por la cual pudiera entrar flujo. Sin embargo, esas casillas exigirían recibir flujo por (17). Esto es imposible. Por tanto, toda casilla usada debe estar conectada con s.

2.6 Modelo completo

El modelo completo puede resumirse como

$$\begin{aligned}
\text{mín} \quad & 0 \\
\text{s.a.} \quad & (4) - (17), \\
& y_v \in \{0,1\} \quad \forall v \in V, \\
& x_e \in \{0,1\} \quad \forall e \in E, \\
& f_{uv} \geq 0 \qquad \forall (u,v) \in A.
\end{aligned}$$

La abreviación “s.a.” significa “sujeto a”. En otras palabras, el solver busca valores para las variables que satisfagan todas las restricciones. Si los encuentra, se obtiene una solución válida del puzzle. Si no los encuentra, la instancia se declara infactible para este modelo.

3 Interpretación del Modelo

3.1 Por qué el modelo produce un camino

El modelo no dice directamente "construir un camino" con una sola restricción. En vez de eso, combina varias familias de restricciones que, juntas, obligan a que la solución sea un camino.

Primero, los terminales s y t tienen grado uno:

$$\sum_{e \in \delta(s)} x_e = 1, \qquad \sum_{e \in \delta(t)} x_e = 1.$$

Esto significa que son los dos extremos de la vía.

Segundo, toda casilla usada que no sea terminal tiene grado dos:

$$\sum_{e \in \delta(v)} x_e = 2y_v.$$

Una casilla de grado dos no puede ser una rama. Solo puede tener una conexión de entrada y una conexión de salida. Si las dos conexiones están en lados opuestos, la pieza es recta. Si están en lados perpendiculares, la pieza es una curva.

Tercero, las restricciones de flujo obligan a que todas las casillas usadas estén conectadas con s. Por tanto, no puede existir una parte de la vía separada del camino principal.

La combinación de estas tres ideas es la clave:

- grado uno en los terminales;
- grado dos en las casillas internas;
- una sola componente conexa.

Con esas condiciones, la estructura seleccionada no puede ser otra cosa que un camino desde s hasta t.

3.2 Qué configuraciones inválidas quedan eliminadas

El modelo elimina varios tipos de errores típicos.

3.2.1 Ramas

Una rama aparece cuando una casilla se conecta con tres o más casillas vecinas. En un grafo, eso corresponde a un vértice de grado al menos tres. El modelo lo prohíbe porque las casillas internas usadas tienen grado exactamente dos, y los terminales tienen grado exactamente uno.

3.2.2 Casillas aisladas

Una casilla aislada sería una casilla marcada como vía, pero sin conexiones. Si $y_v = 1$ y v no es terminal, la restricción

$$\sum_{e \in \delta(v)} x_e = 2y_v$$

obliga a tener exactamente dos conexiones. Por tanto, una casilla usada no puede quedar sola.

3.2.3 Ciclos aislados

Un ciclo aislado puede respetar las restricciones de grado, porque cada casilla del ciclo tendría grado dos. Por eso las restricciones locales no bastan. La formulación de flujo resuelve este problema: un ciclo aislado no puede recibir flujo desde s, así que no puede existir en una solución factible.

3.2.4 Componentes desconectadas

Una componente desconectada es un grupo de casillas unidas entre sí, pero separadas del resto de la vía. En el curso, una componente conexa se entiende como un subgrafo conexo maximal. Tracks exige que las casillas usadas formen una única componente conexa. El flujo impone exactamente esa idea, porque cada casilla usada debe ser alcanzable desde s.

3.3 Relación entre las pistas y la geometría

Las pistas de filas y columnas solo cuentan casillas. No dicen cómo se conectan esas casillas. Por ejemplo, dos configuraciones pueden usar el mismo número de casillas en cada fila y columna, pero una puede formar una vía válida y la otra puede estar desconectada.

Por eso el modelo separa dos niveles:

- las variables y_v controlan cuántas casillas se usan;
- las variables x_e controlan cómo se conectan esas casillas.

Las restricciones de filas y columnas usan solamente las variables y_v:

$$\sum_{j \in C} y_{(i,j)} = \rho_i, \qquad \sum_{i \in R} y_{(i,j)} = \gamma_j.$$

Las restricciones de grado y conectividad usan las variables x_e y f_{uv}. Esta separación hace que el modelo sea más claro: primero se decide dónde hay vía, y después se garantiza que esas casillas estén conectadas correctamente.

3.4 Por qué no se necesitan variables de tipo de pieza

Una posible forma de modelar Tracks sería crear una variable para cada tipo de pieza: recta horizontal, recta vertical, curva arriba-derecha, curva arriba-izquierda, etc. Ese enfoque es posible, pero no es necesario.

En este modelo, la forma de la pieza se deduce de las aristas seleccionadas alrededor de la casilla. Si una casilla tiene seleccionadas las aristas izquierda y derecha, entonces es una pieza recta horizontal. Si tiene seleccionadas las aristas arriba y abajo, es una pieza recta vertical. Si tiene seleccionadas una arista vertical y una horizontal, es una curva.

Por tanto, las variables x_e contienen suficiente información para reconstruir la pieza gráfica después. Esto es útil para la implementación futura: el solver puede devolver las aristas seleccionadas, y la parte visual puede traducirlas en dibujos de vías.

3.5 Comentarios sobre otras formulaciones posibles

La formulación con flujo no es la única manera de imponer conectividad. Otra opción consiste en usar restricciones de corte. Para cualquier subconjunto $S \subseteq V \setminus \{s\}$, se puede imponer una

restricción del tipo

$$\sum_{e \in \delta(S)} x_e \geq y_v \qquad \forall v \in S.$$

La idea es la siguiente: si dentro de S hay una casilla usada, entonces debe existir al menos una arista seleccionada que salga de S. Si no existe tal arista, entonces S estaría desconectado del origen.

Estas restricciones son matemáticamente correctas, pero hay muchísimos subconjuntos S. Por eso, en la práctica, no se suelen escribir todas desde el principio. Se pueden añadir progresivamente mediante cortes o restricciones perezosas cuando el solver encuentra una solución desconectada. Esta idea es parecida a las restricciones de eliminación de subtours en problemas de rutas.

Para este proyecto, el modelo con flujo es una primera formulación más directa. Tiene más variables continuas, pero es más fácil de explicar, implementar y verificar. Además, expresa claramente la propiedad más importante: todas las casillas usadas deben estar conectadas con el terminal inicial.

3.6 Resumen final del modelado

El puzzle Tracks se modela como un problema de factibilidad sobre un grafo de grilla. Las casillas son vértices, las adyacencias ortogonales son aristas, y la vía buscada es un subgrafo seleccionado.

El modelo usa:

- variables binarias y_v para decidir qué casillas contienen vía;
- variables binarias x_e para decidir qué conexiones entre casillas se usan;
- variables de flujo f_{uv} para certificar que no hay componentes desconectadas.

Cada familia de restricciones tiene un papel preciso. Las pistas de filas y columnas controlan el número de casillas usadas. Las restricciones de coherencia impiden conectar casillas vacías. Las restricciones de grado imponen la forma local de una vía sin ramas. Las restricciones de información fija respetan las pistas pre-rellenadas. Las restricciones de flujo garantizan que toda la solución forme una sola componente conexa.

Así, una solución factible del modelo corresponde exactamente a una solución válida del puzzle bajo las reglas descritas. Este modelo será la base matemática para la implementación posterior del solver en Python.